



Flink state evolution

Flink serialization mechanisms,
and how to evolve the state's structure

September 29th, 2026

Speakers



Arnaud Daroussin

Senior Data Engineer

GitHub: github.com/arnaud-daroussin
LinkedIn: [linkedin.com/in/arnaud-daroussin-6a82b016a/](https://www.linkedin.com/in/arnaud-daroussin-6a82b016a/)



Gaël Renoux

Staff Engineer

GitHub: github.com/gaelrenoux
& github.com/gaelrenoux-datadome
LinkedIn: [linkedin.com/in/gaël-renoux-71956013/](https://www.linkedin.com/in/gaël-renoux-71956013/)



Scraping



Account
Takeover



Credential
Stuffing



Payment



Fake Account
Creation



Scalper Bots



Denial of
Service



Vulnerability
scanning



Ad/Click
Fraud



Skewed
Analytics

Flink at DataDome

300K

Events/second

7

Flink jobs

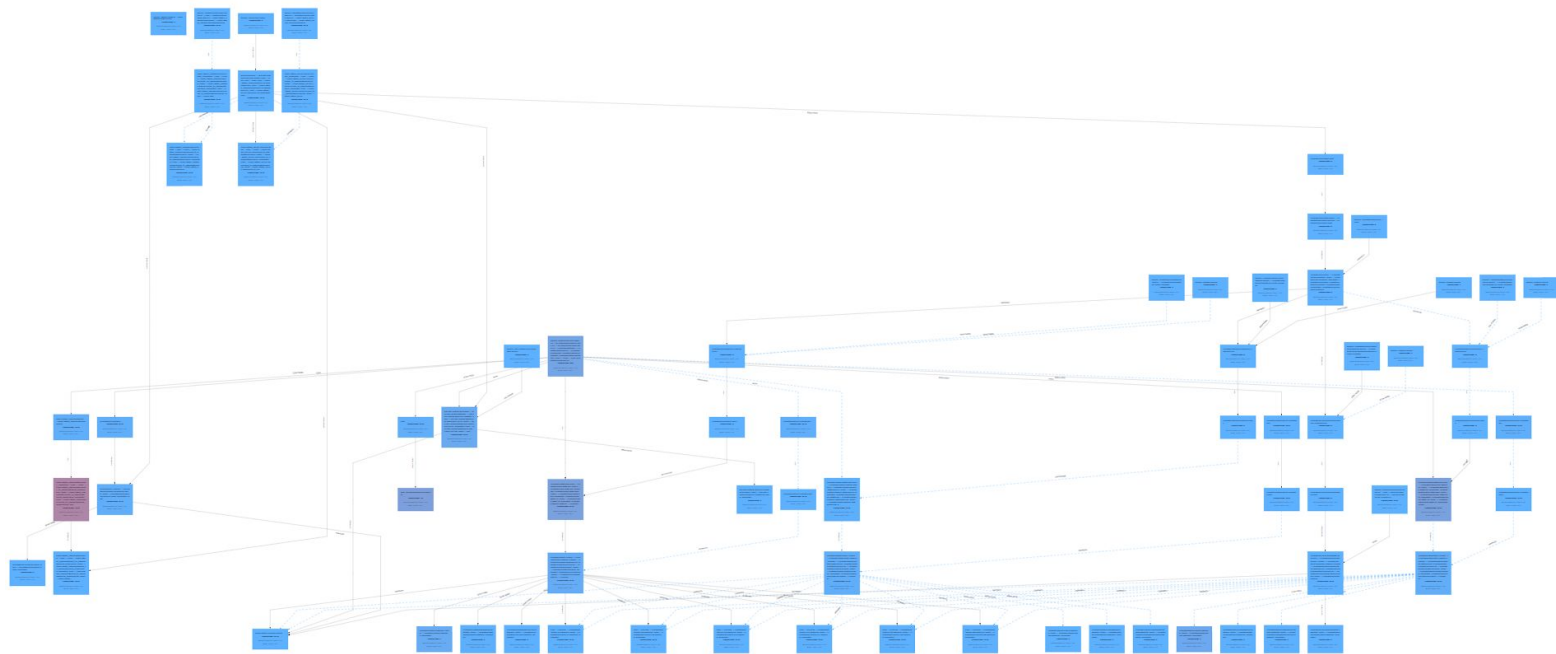
~330

Task-managers

~30

Deployments/month

Flink at DataDome



The problem

Upgrading...

2026-09-14 14:10:43

✓ java.lang.Exception: Except

at org.apache.flink.str

at org.apache.flink.str

at org.apache.flink.str

at org.apache.flink.str

at org.apache.flink.str

at org.apache.flink.str

at org.apache.flink.str

at org.apache.flink.str

at org.apache.flink.str

at org.apache.flink.str

at org.apache.flink.str

at org.apache.flink.str

at org.apache.flink.str

at org.apache.flink.str

at org.apache.flink.str

at org.apache.flink.str

at org.apache.flink.str

at org.apache.flink.str

at org.apache.flink.str

at org.apache.flink.str

at org.apache.flink.str

at org.apache.flink.str

at org.apache.flink.str

at org.apache.flink.str

at org.apache.flink.str

at org.apache.flink.str

Caused by: java.lang.RuntimeException: Error while getting state

at org.apache.flink.runtime.state.DefaultKeyedStateStore.getState(DefaultKeyedStateStore.java:62)

at org.apache.flink.streaming.api.operators.StreamingRuntimeContext.getState(StreamingRuntimeContext.java:203)

at co.datadome.sherlock.newVerdicts.rules.IndexBuildFunction.buildPostponedCountStates\$lzycompute(IndexBuildFunction.scala:74)

at co.datadome.sherlock.newVerdicts.rules.IndexBuildFunction.buildPostponedCountState(IndexBuildFunction.scala:74)

at co.datadome.sherlock.newVerdicts.rules.IndexBuildFunction.getBuildPostponedCount(IndexBuildFunction.scala:76)

at co.datadome.sherlock.newVerdicts.rules.IndexBuildFunction.onTimer(IndexBuildFunction.scala:91)

at org.apache.flink.streaming.api.operators.KeyedProcessOperator.invokeUserFunction(KeyedProcessOperator.java:294)

at org.apache.flink.streaming.api.operators.KeyedProcessOperator.onProcessingTime(KeyedProcessOperator.java:294)

at org.apache.flink.streaming.runtime.tasks.StreamTask.invokeProcessingTimeCall(StreamTask.java:712)

at org.apache.flink.streaming.runtime.tasks.StreamTask.restore(StreamTask.java:712)

at org.apache.flink.streaming.runtime.taskmanager.Task.runWithSystemExitMonitoring(Task.java:958)

at org.apache.flink.streaming.runtime.taskmanager.Task.restoreAndInvoke(Task.java:927)

at org.apache.flink.streaming.runtime.taskmanager.Task.doRun(Task.java:751)

at org.apache.flink.streaming.runtime.taskmanager.Task.run(Task.java:566)

at java.base/java.lang.Thread.run(Thread.java:1583)

Caused by: org.apache.flink.util.StateMigrationException: For heap backends, the

flink.api.common.typeutils.base.IntSerializer@3df6911a).

at org.apache.flink.runtime.state.heap.HeapKeyedStateBackend.tryRegister(HeapKeyedStateBackend.java:1583)

at org.apache.flink.runtime.state.heap.HeapKeyedStateBackend.createOrRestore(HeapKeyedStateBackend.java:1583)

at org.apache.flink.runtime.state.heap.HeapKeyedStateBackend.createOrRestore(HeapKeyedStateBackend.java:1583)

at org.apache.flink.runtime.state.heap.HeapKeyedStateBackend.createOrRestore(HeapKeyedStateBackend.java:1583)

at org.apache.flink.runtime.state.heap.HeapKeyedStateBackend.createOrRestore(HeapKeyedStateBackend.java:1583)

at org.apache.flink.runtime.state.heap.HeapKeyedStateBackend.createOrRestore(HeapKeyedStateBackend.java:1583)

at org.apache.flink.runtime.state.heap.HeapKeyedStateBackend.createOrRestore(HeapKeyedStateBackend.java:1583)

at org.apache.flink.runtime.state.heap.HeapKeyedStateBackend.createOrRestore(HeapKeyedStateBackend.java:1583)

at org.apache.flink.runtime.state.heap.HeapKeyedStateBackend.createOrRestore(HeapKeyedStateBackend.java:1583)

at org.apache.flink.runtime.state.heap.HeapKeyedStateBackend.createOrRestore(HeapKeyedStateBackend.java:1583)

at org.apache.flink.runtime.state.heap.HeapKeyedStateBackend.createOrRestore(HeapKeyedStateBackend.java:1583)

at org.apache.flink.runtime.state.heap.HeapKeyedStateBackend.createOrRestore(HeapKeyedStateBackend.java:1583)

at org.apache.flink.runtime.state.heap.HeapKeyedStateBackend.createOrRestore(HeapKeyedStateBackend.java:1583)

at org.apache.flink.runtime.state.heap.HeapKeyedStateBackend.createOrRestore(HeapKeyedStateBackend.java:1583)

at org.apache.flink.runtime.state.heap.HeapKeyedStateBackend.createOrRestore(HeapKeyedStateBackend.java:1583)

at org.apache.flink.runtime.state.heap.HeapKeyedStateBackend.createOrRestore(HeapKeyedStateBackend.java:1583)

at org.apache.flink.runtime.state.heap.HeapKeyedStateBackend.createOrRestore(HeapKeyedStateBackend.java:1583)

at org.apache.flink.runtime.state.heap.HeapKeyedStateBackend.createOrRestore(HeapKeyedStateBackend.java:1583)

at org.apache.flink.runtime.state.heap.HeapKeyedStateBackend.createOrRestore(HeapKeyedStateBackend.java:1583)

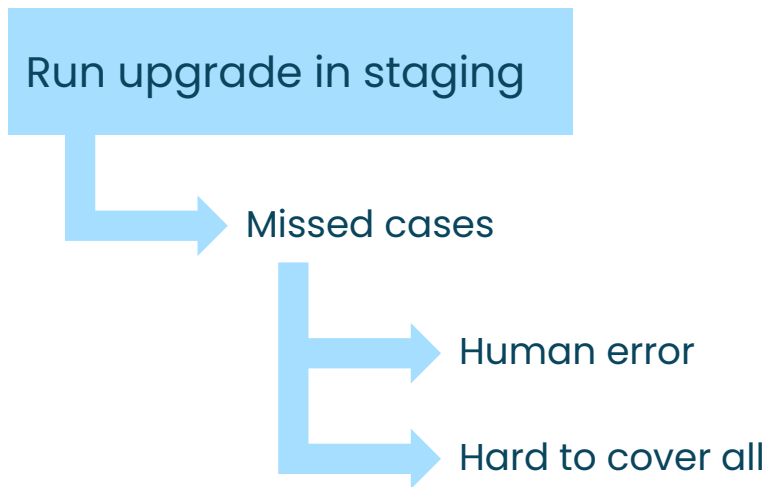
at org.apache.flink.runtime.state.heap.HeapKeyedStateBackend.createOrRestore(HeapKeyedStateBackend.java:1583)

at org.apache.flink.runtime.state.heap.HeapKeyedStateBackend.createOrRestore(HeapKeyedStateBackend.java:1583)

at org.apache.flink.runtime.state.heap.HeapKeyedStateBackend.createOrRestore(HeapKeyedStateBackend.java:1583)



« Just test it ! »



Early solution: State signature

State evolution $\Rightarrow$ Operator UID reset



[Flink Forward 2020 Handling frequent changes in state structure](#)



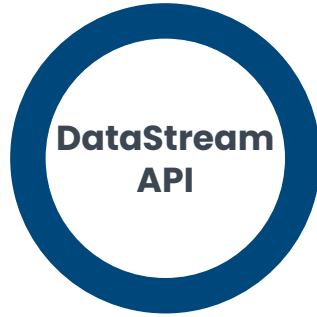
Not a solution: Flink's State Processor API

Upgrade the savepoint before redeploying

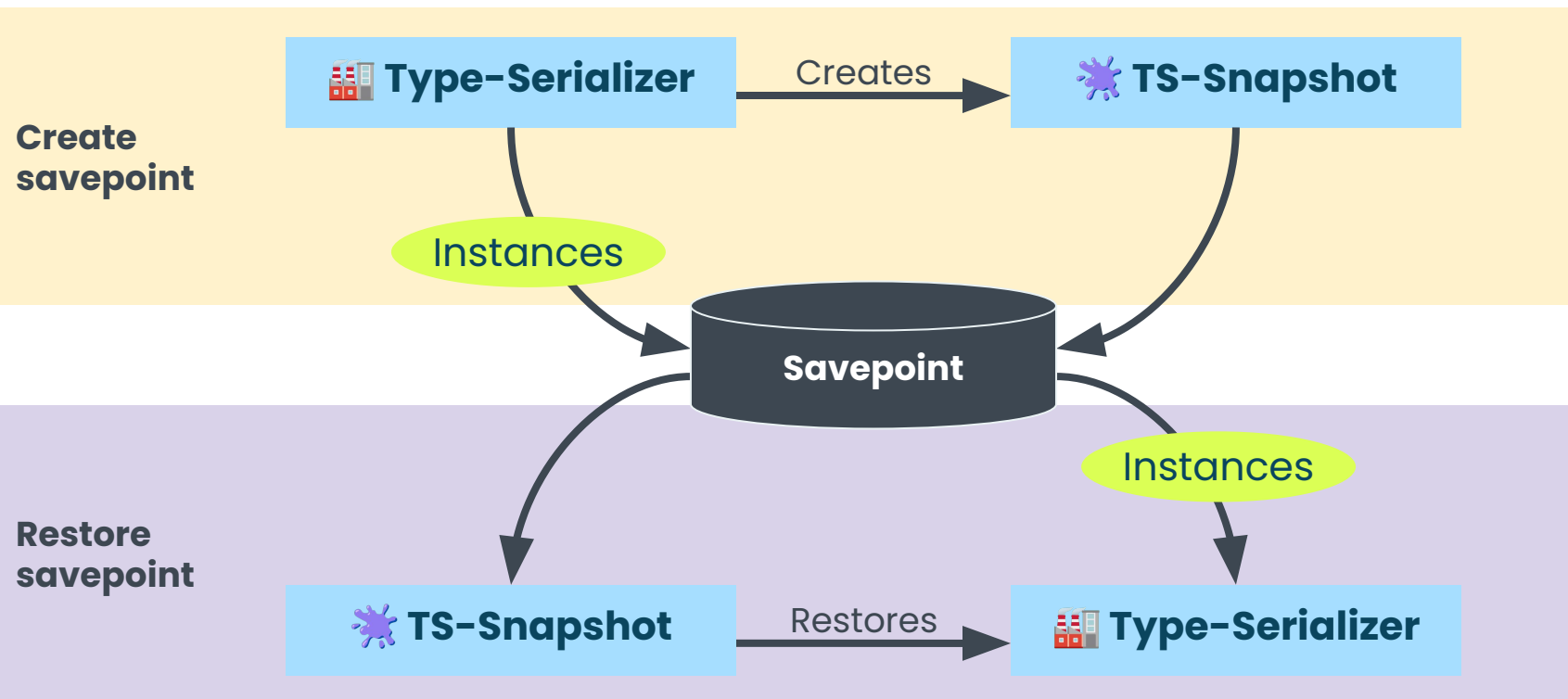


How serialization works

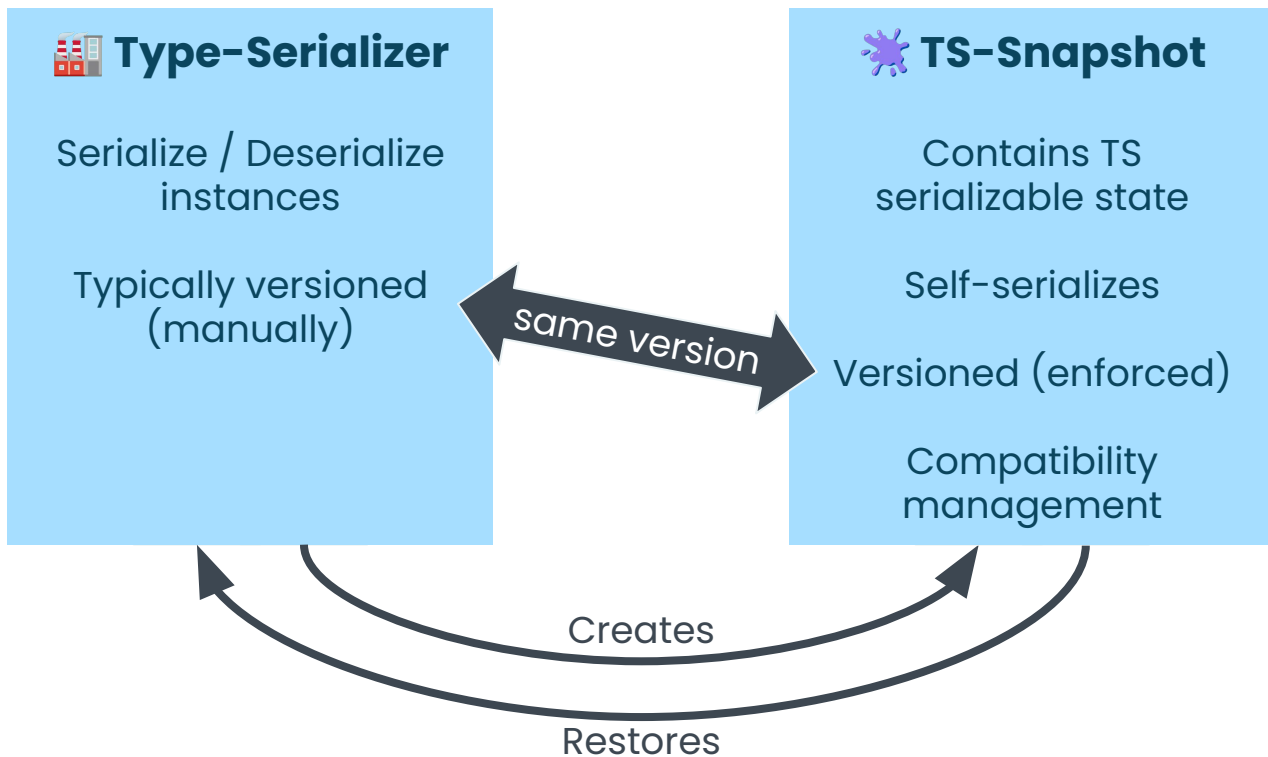
Caveats



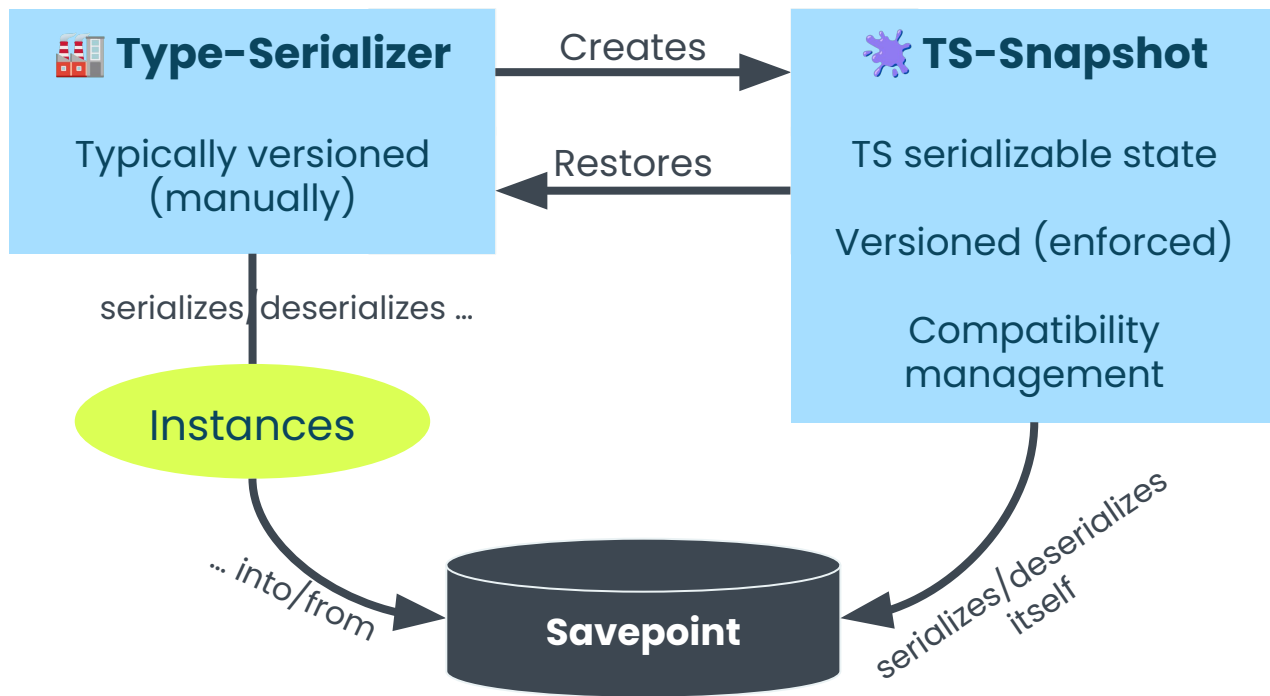
Type-Serializer and Type-Serializer-Snapshot



Type-Serializer and Type-Serializer-Snapshot



Type-Serializer and Type-Serializer-Snapshot



When creating a savepoint

Register TS

```
DESC = new ValueStateDescriptor<Stats>(  
    "stats",  
    new StatsSerializer()  
);
```

My job

Flink

Serialize TSS

```
snap = ser.snapshotConfiguration();
```

```
out.writeUTF(snap.getClass().getName());  
out.writeInt(snap.getCurrentVersion());  
snap.writeSnapshot(out);
```

Serialize state

```
ser.serialize(...);
```

Passes the version number

No need to serialize the version manually

Uses the latest version by default

When restarting from a savepoint

Requires a no-arg constructor !

Restore TSS then TS

```
className = in.readUTF();  
clazz = Class.forName(className, ...);  
oldSnap = clazz.newInstance();  
version = in.readInt();  
oldSnap.readSnapshot(version, in, ...);
```

```
oldSer = oldSnap.restoreSerializer();
```

Passes the version number

Register TS and TSS

```
DESC = new ValueStateDescriptor<Stats>(  
    "stats",  
    new StatsSerializer() // newSer  
);
```

```
newSnap = newSer.snapshotConfiguration();
```

May be different classes

Check compatibility

```
newSnap.resolveSchemaCompatibility(oldSnap);
```

Deserialize state

```
ser.deserialize(...);
```

We'll get back to that one

Heap vs RocksDB Backends

	Heap	RocksDB
Stores:	Java instances	Serialized bytes
Serializes:	On savepoint	On state update



Creating a savepoint



Heap Backend

Get registered TS

Create TSS

Serialize TSS

Serialize instance

RocksDB Backend

Get registered TS

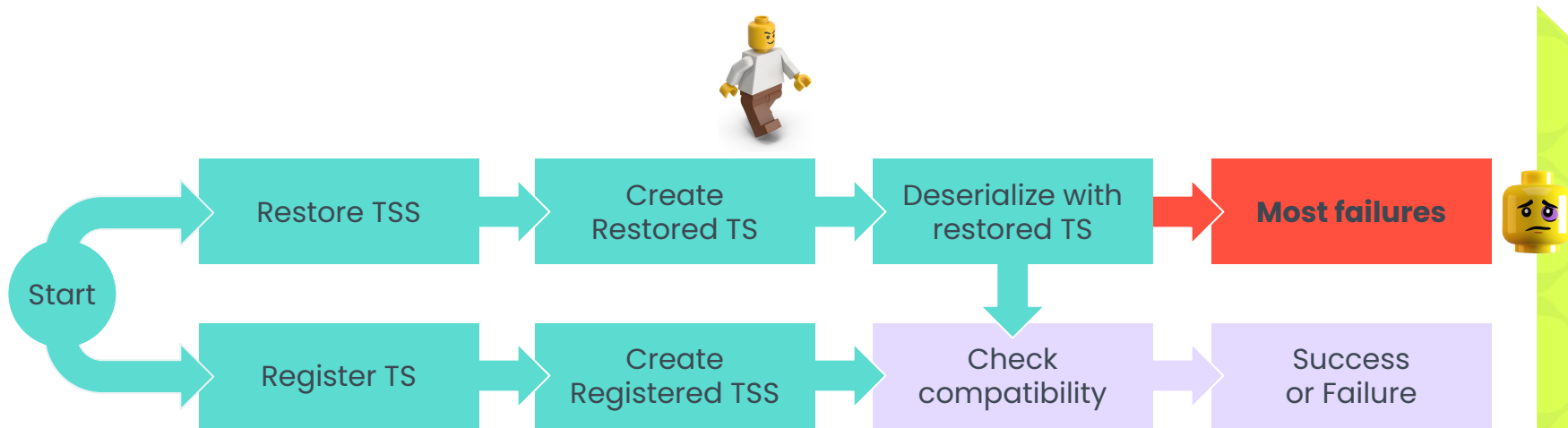
Create TSS

Serialize TSS

Copy bytes

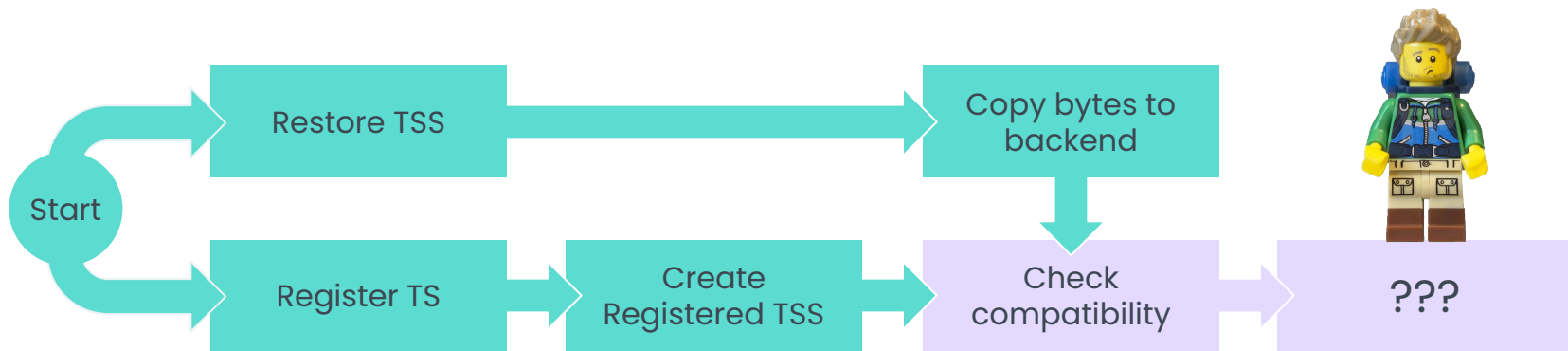
Restoring a savepoint

Heap
Backend



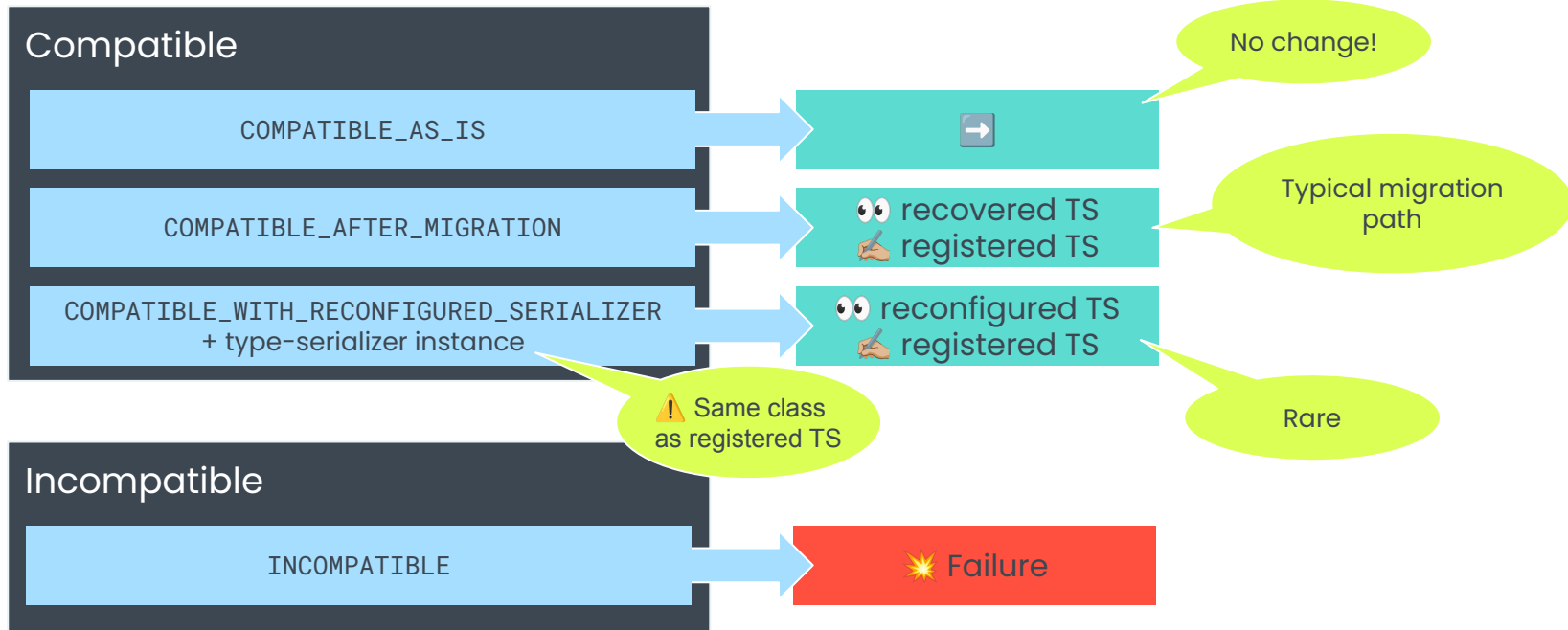
Restoring a savepoint

RocksDB
Backend



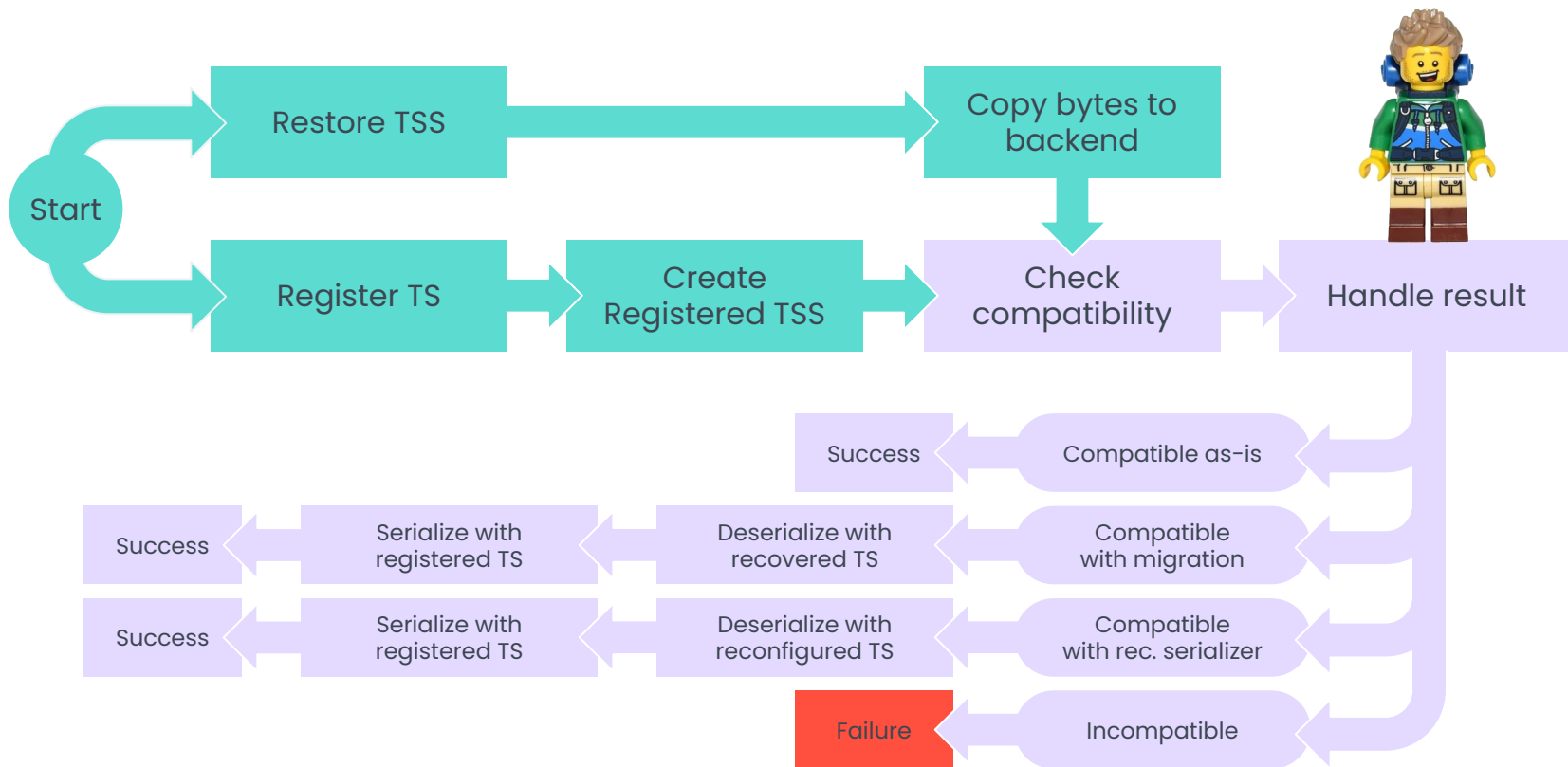
Checking compatibility

```
snapNew.resolveSchemaCompatibility(snapOld);
```



Restoring a savepoint

RocksDB
Backend



Learn more



State Schema Evolution
Custom Serialization for Managed State

```
HeapKeyedStateBackend.tryRegisterStateTable(...)
```

```
RocksDBKeyedStateBackend.updateRestoredStateMetaInfo(...)
```



Evolution use cases

POJO serializer: supported evolutions



Remove fields



Add fields



Reorder fields



From POJO to custom serializer

```
class Count {  
    private String ip;  
    private long totalCount;  
    private int errorCount;  
    ... // no-arg constructor + getters + setters  
}
```

Default PojoSerializer

```
COUNT_DESCRIPTOR = new ValueStateDescriptor<Count>( "count", Count.class);
```

Custom Serializer

```
COUNT_DESCRIPTOR = new ValueStateDescriptor<Count>( "count", new CountSerializer());
```

Change the type of a field

Restored savepoint

"CountSerializerSnapshot"

version = 1

Some Count fields

int errorCount

CountSerializer

```
Count deserialize(Count count, DataInputView source) {  
    ...  
    if (this.version == V1) {  
        count.setErrorCount(source.readInt());  
    } else {  
        count.setErrorCount(source.readLong());  
    }  
    return count;  
}
```

Read as a int

CountSerializerSnapshot

```
TSSchemaCompatibility<Count>  
resolveSchemaCompatibility(TSS<Count> restored) {  
    if (this.version == V2 &&  
        restored.version == V1) {  
        return compatibleAfterMigration();  
    } else ...  
}
```

Next savepoint

"CountSerializerSnapshot"

version = 2

Some Count fields

long errorCount

*Both must be true to
trigger the migration path*

Rename a state class

Restored savepoint

"CountSerializerSnapshot"

version = 2

Count fields

CountSerializerSnapshot

```
TypeSerializer<Count> restoreSerializer() {  
    return new StatsSerializer(version);  
}
```

StatsSerializerSnapshot

```
TypeSerializerSchemaCompatibility<Stats>  
    resolveSchemaCompatibility(TSS<Stats> restored) {  
  
    if (restored instanceof StatsSerializerSnapshot ||  
        restored instanceof CountSerializerSnapshot) {  
        return compatibleAsIs();  
    }  
    ...  
}
```

Next savepoint

"StatsSerializerSnapshot"

version = 2

Stats fields

Accept old and new name

Work-in-progress: evolution annotations

```
@version(3)
@deletedFields(since = 2, "timestamp")
@renamed(since = 2, "Count")
case class Stats(
  ip: String,
  @renamed(since = 3, "total") totalCount: Long,
  @transformed(since = 2, mapLongToInt) errorCount: Int,
  @added(since = 3) redirectCount: Long = 0
)

def mapLongToInt(value: Long): Int = value.toInt
```

Coming soon... in flink-scala-api



Unit testing

State validation

Run test (1st time)

Create savepoint in src/test/resources

Committed to Git

Re-run test

Compare with existing savepoint



No diff



Compatible changes



Incompatible

*Test is failed (yellow)
Might be just content!
Check if expected*

Test in error (red)

Using Flink's
test harnesses
(AbstractStreamOperator
TestHarness)

With a wrapper because 🤔

Coming soon: state validation harness fixture

```
trait Fixture extends FixtureTestSuite {  
  
  type FixtureParam <: Harness[_]_  
  
  def makeHarness(savepointFilePath: Option[String] = None): FixtureParam  
  
  override def withFixture(test: OneArgTest): Outcome = {  
    val harness: FixtureParam = makeHarness()  
    try {  
      val outcome = withFixture(test.toNoArgTest(harness))  
      if (outcome.isSucceeded) {  
        val validation = validate(harness, test.name, makeHarness(Some(_)))  
        handleValidation(validation)  
        outcome  
      } else outcome  
    } finally harness.close()  
  }  
}
```

Implementation in ScalaTest

```
class ProcessFunctionHarnessTest extends TestSuite with HarnessStateValidationFixture
{
  override def makeHarness(savepointFilePath: Option[String] = None): Harness =
    ProcessFunctionHarness(new MyFunction, savepointFilePath)

  "Process message and read" in { harness =>
    harness.process(42)
    harness.process(100)
    harness.getOutput should be(List(42, 100))
  }
}
```

Coming soon... in flink-scala-api

Thank you! Any Questions?

<http://datadome.co>

